Resumen del proceso de implementación de un sistema de identificación de tortugas

Durante este preproyecto se construyó en **Windows** un entorno de trabajo local en Python enfocado en el desarrollo de un sistema para el **reconocimiento individual de tortugas marinas** a partir de imágenes de sus cabezas y aletas. La meta principal es **dar seguimiento a individuos en el tiempo**, registrando sus patrones únicos y permitiendo identificar a la misma tortuga en avistamientos posteriores, incluso en diferentes temporadas.

El proceso inició con la **preparación del entorno local**, creando una carpeta de proyecto estructurada y un entorno virtual (`.venv`) en Visual Studio Code. Posteriormente se instalaron librerías críticas: **PyTorch**, **Torchvision**, **Transformers**, **FAISS** y **Pillow**. Durante la instalación se presentaron retos, especialmente con dependencias de GPU, por lo que inicialmente se optó por ejecutar el sistema en versión CPU con PyTorch 2.6.0, garantizando compatibilidad con la librería `transformers`.

Con el entorno estable, se configuró el script principal `turtle_id_demo.py`, el cual cumple tres funciones esenciales:

1. **Enrolamiento**: agregar imágenes de individuos conocidos a la base (ejemplo: `t001`, `t002`, etc.), generando representaciones vectoriales (embeddings) con el modelo **CLIP (openai/clip-vit-base-patch32)** de Hugging Face.

2. **Consulta**: comparar una nueva fotografía contra la base, retornando los individuos más similares y un valor de similitud para decidir si corresponde a una tortuga ya registrada o si se trata de un nuevo ejemplar.

3. **Estadísticas**: revisar el número de individuos e imágenes procesadas, confirmando el crecimiento de la base de datos.

La fase práctica consistió en **procesar más de 13,000 imágenes pertenecientes a 387 individuos** diferentes, organizadas en carpetas (`t001`, `t002`, …, `t522`). Los resultados confirmaron el correcto enrolamiento de las imágenes y la construcción de un índice de búsqueda eficiente con FAISS.

Aunque el sistema ya es funcional, se identificaron varias áreas de mejora:

- **Uso de GPU**: instalar correctamente los controladores CUDA y librerías VC++ para aprovechar la aceleración por hardware en lugar de depender únicamente de CPU, reduciendo drásticamente los tiempos de procesamiento.

- **Recorte automático de regiones de interés**: integrar modelos de detección (como YOLO) para aislar cabeza o aleta antes de generar embeddings, aumentando la precisión en la identificación.

- **Modelos especializados (siamesas o ArcFace)**: entrenar redes neuronales específicas con las imágenes recolectadas para superar las limitaciones de un modelo genérico como CLIP, logrando mayor robustez en condiciones de iluminación, ángulo y ruido.

- **Interfaz gráfica amigable**: empaquetar el proyecto en una aplicación de escritorio con botones y etapas claras, de manera que investigadores o voluntarios puedan enrolar o consultar individuos sin necesidad de ejecutar comandos manuales.
-

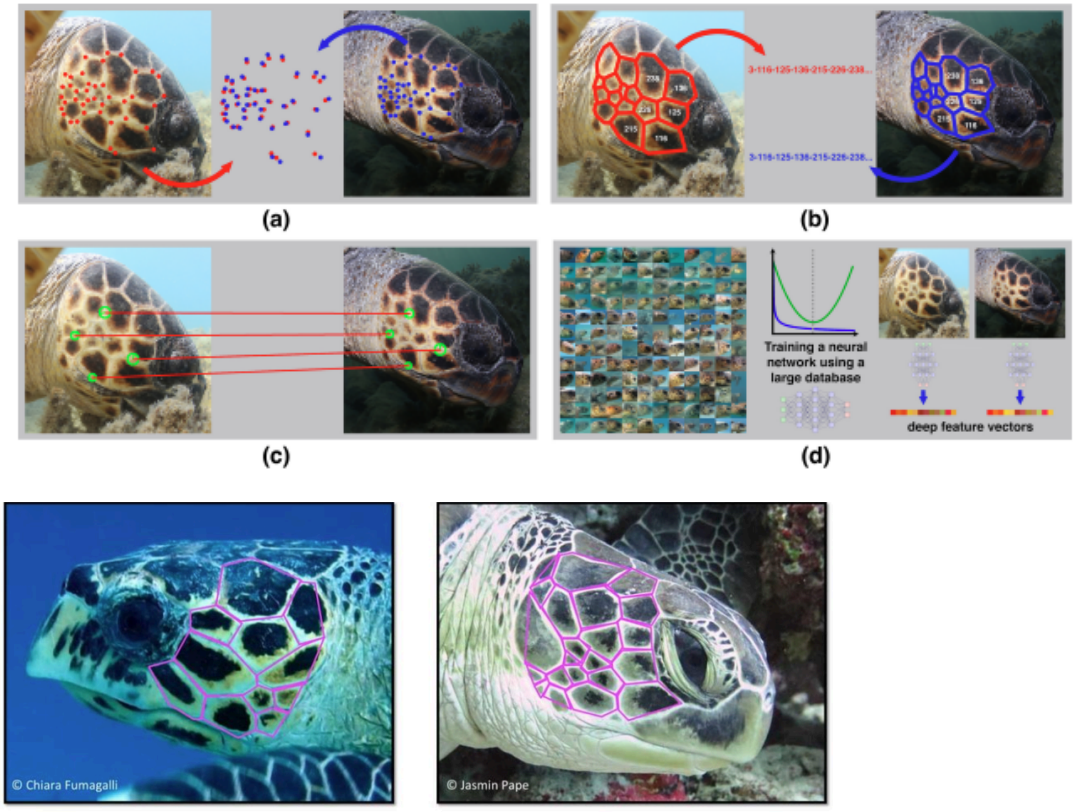
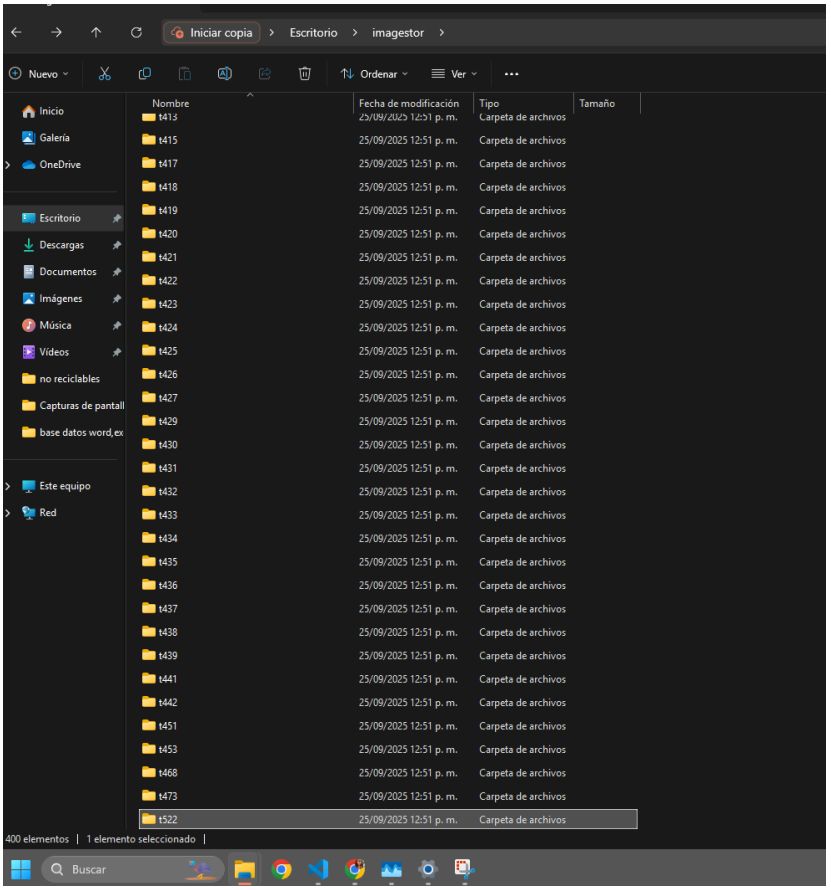
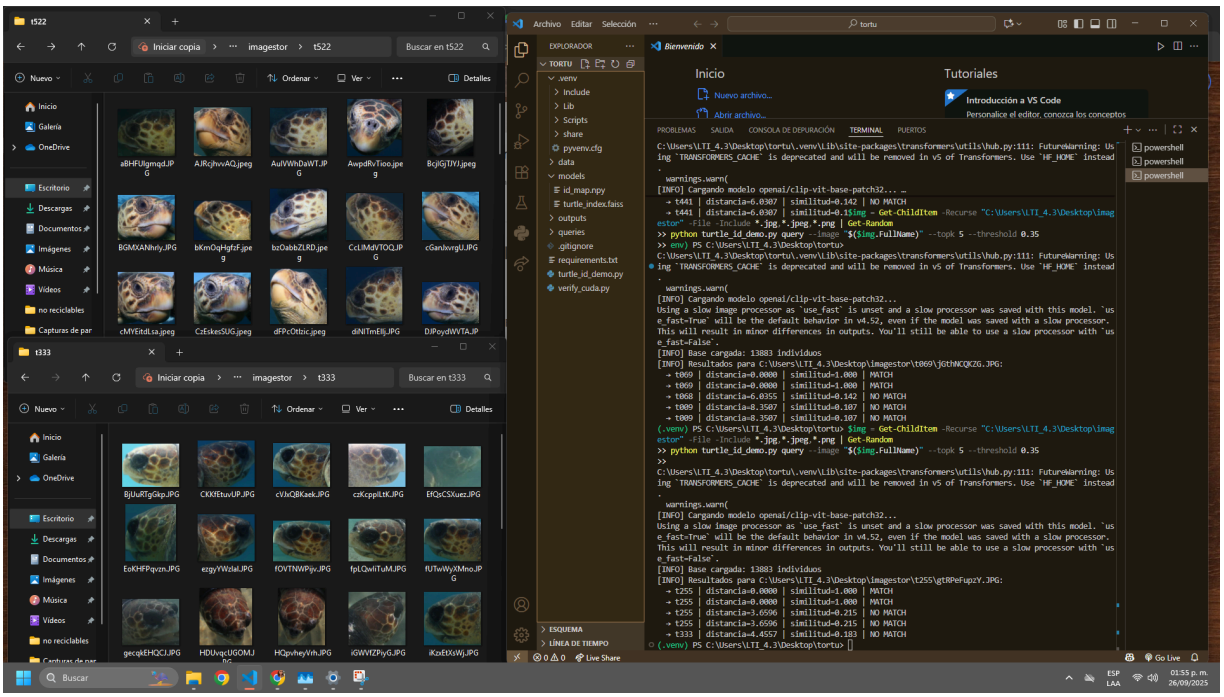
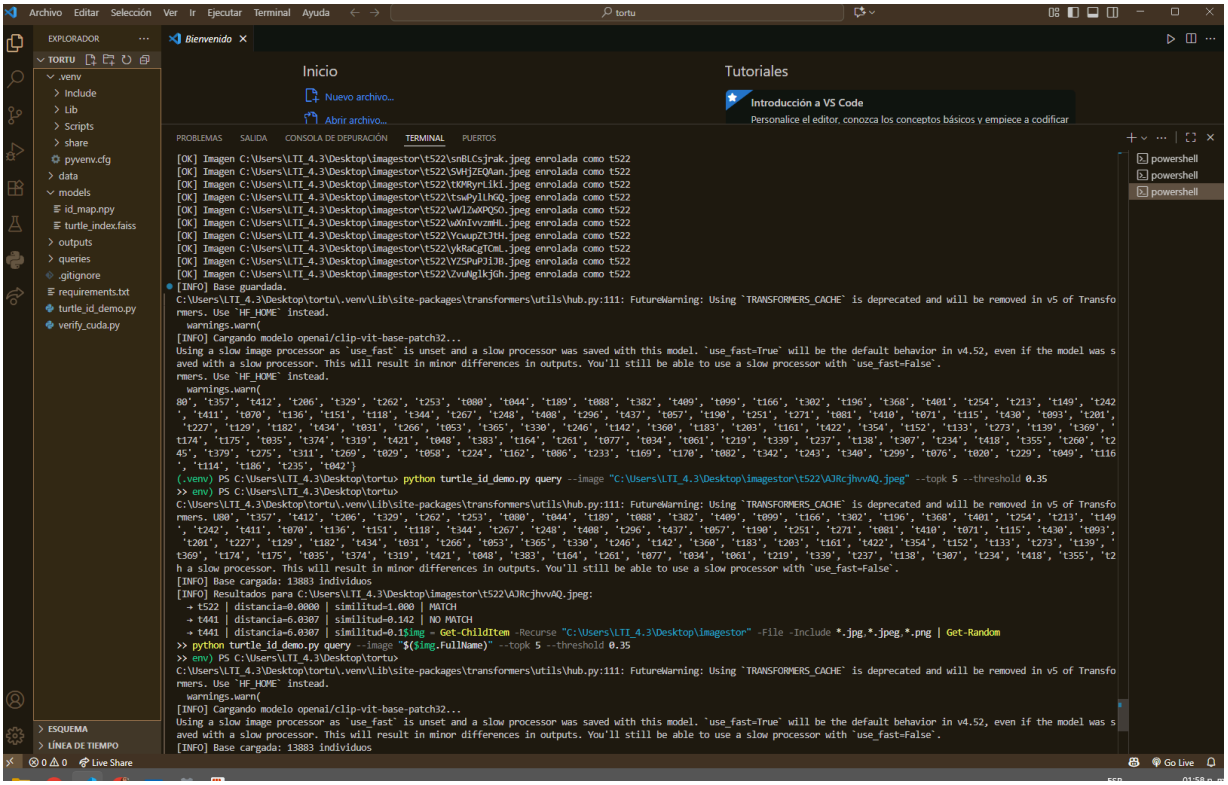


Fig. 1. Turtles have a unique pattern of facial scutes. L & R sides are asymmetrical. Area used for ID based on Jean et al. (2010)







En conclusión, el trabajo realizado logró sentar las bases técnicas para un **sistema de identificación individual de tortugas marinas**, con un flujo completo de enrolamiento, consulta y almacenamiento de imágenes. A mediano plazo, con las optimizaciones pendientes, se busca disponer de una herramienta robusta y eficiente para el **monitoreo y conservación de especies marinas**, aplicable en proyectos científicos y de protección ambiental.

El CLIP de OpenAI (Contrastive Language–Image Pretraining) es un modelo multimodal de visión y lenguaje que OpenAI entrenó en 2021 con 400 millones de pares de imágenes y descripciones de internet, diseñado para alinear los embeddings de las imágenes con los de los textos que las describen. Funciona con dos codificadores, uno de imágenes (como ResNet o Vision Transformer) y otro de texto (basado en Transformer), que generan vectores en un mismo espacio latente de forma que, por ejemplo, la imagen de una tortuga y la palabra “tortuga” producen vectores cercanos entre sí. Aunque CLIP no fue entrenado específicamente en tortugas, resulta útil en tu proyecto porque puede representar rasgos visuales generales como colores, formas y patrones, actuando como un extractor universal de características que convierte cada foto en un embedding numérico de 512 dimensiones; estos embeddings luego se comparan con FAISS para determinar si una nueva foto se parece a otra ya registrada. A diferencia de una red siamesa, que sería entrenada con tu dataset y se especializaría en distinguir tortugas con mayor precisión, CLIP es un modelo generalista ya preentrenado que funciona directamente, lo que lo hace una opción rápida para probar el flujo de identificación, con la posibilidad de ser sustituido más adelante por una siamesa que brinde mejores resultados en el seguimiento de individuos.

Actualmente tenemos un sistema basado en CLIP (openai/clip-vit-base-patch32) como extractor de características: cada imagen se transforma en un vector numérico (embedding) y, con FAISS, comparamos la similitud entre embeddings para decidir si una foto nueva se parece a alguna tortuga registrada. Esto es rápido de implementar y no requiere un entrenamiento complejo, aunque presenta la limitación de que CLIP fue entrenado con imágenes generales y no específicamente con tortugas, lo cual puede reducir la precisión en casos difíciles. Una alternativa más especializada serían las redes neuronales siamesas, que consisten en dos redes idénticas con pesos compartidos alimentadas con pares de imágenes (por ejemplo, cabeza de la tortuga A frente a cabeza de la tortuga B); la red aprende a producir embeddings cercanos si son del mismo individuo y lejanos si son distintos, utilizando funciones de pérdida como contraste o triplet loss. En la práctica, la diferencia es clara: el esquema actual con CLIP + FAISS permite identificación rápida con embeddings genéricos, mientras que una siamesa entrenada en tu dataset generaría embeddings especializados en cabezas o aletas de tortugas, ofreciendo mayor precisión y menor tasa de falsos positivos. Para integrar siamesas en el proyecto será necesario preparar el dataset con pares positivos y negativos, implementar la arquitectura en PyTorch (por ejemplo, con ResNet50 como base), entrenar durante varias épocas hasta estabilizar la pérdida, guardar el modelo entrenado (siamese_turtle.pth) y reemplazar CLIP en turtle_id_demo.py por este modelo especializado. En resumen, lo que tenemos ahora es una “versión A”, un sistema funcional con un modelo general ya operativo, y la implementación de siamesas sería la “versión B”, que requiere mayor trabajo de entrenamiento pero ofrece un sistema mucho más preciso y adaptado al problema real de identificación de tortugas individuales.

Resumen de la Fase 2: Implementación de Redes Neuronales Siamesas y Optimización de Procesamiento para Identificación de Tortugas

La segunda fase del proyecto se orienta a superar las limitaciones de la versión inicial, que empleaba CLIP de OpenAI como extractor de características genéricas para identificar tortugas marinas. Si bien esta primera aproximación permitió obtener un sistema funcional capaz de comparar imágenes mediante embeddings y FAISS, la precisión no resulta suficiente cuando se trata de reconocer individuos en un escenario ecológico real, donde los patrones de cabeza y aletas presentan gran variabilidad y se requiere un nivel de confianza superior para su uso en investigación. La propuesta de esta fase es implementar redes neuronales siamesas entrenadas específicamente con el dataset recopilado durante más de cinco años en el proyecto de ecología del hotel Moon Palace, con el fin de crear un sistema de identificación especializado, robusto y escalable.

Redes Neuronales Siamesas como núcleo del sistema

Una red siamesa es una arquitectura que emplea dos redes idénticas con pesos compartidos, diseñadas para recibir pares de imágenes y aprender una función de similitud. En este caso, el entrenamiento se realizará con pares positivos (dos imágenes de la misma tortuga) y pares negativos (imágenes de tortugas diferentes). Mediante funciones de pérdida como contrastive loss o triplet loss, la red aprende a mapear imágenes del mismo individuo en vectores cercanos dentro de un espacio latente y a separar aquellos correspondientes a individuos distintos. Este enfoque permite que los embeddings generados estén optimizados exclusivamente para el dominio de patrones de tortugas, aumentando la precisión y reduciendo falsos positivos frente al modelo genérico utilizado en la fase inicial.

Para implementar la siamesa se propone utilizar PyTorch, con una arquitectura basada en ResNet50 como red troncal (backbone). Este modelo ofrecerá un balance entre precisión y eficiencia en el entrenamiento. El resultado final será un modelo entrenado y guardado en un archivo (`siamese_turtle.pth`), que sustituirá a CLIP en el flujo actual, generando embeddings específicos para tortugas. De esta manera, el sistema podrá recibir una nueva imagen desde un cuaderno Jupyter, extraer su embedding especializado y compararlo rápidamente con la base de datos para identificar al individuo con alta fiabilidad.

Optimización del hardware local

La primera fase evidenció una limitación en el tiempo de procesamiento, debido a que se trabajó con un entorno local configurado en CPU. Aunque la PC utilizada (Intel Core i7 de 13ª generación, 32 GB de RAM y GPU de 16 GB) ofrece recursos de alto rendimiento, la instalación de PyTorch se realizó en modo CPU-only, lo cual anuló la capacidad de la tarjeta gráfica dedicada. En esta fase, se establecerá la instalación correcta de PyTorch con soporte CUDA, permitiendo aprovechar la GPU local para acelerar tanto la etapa de entrenamiento como la de inferencia.

Adicionalmente, se considera la creación de un cluster local, es decir, la interconexión de varios equipos con GPU para repartir el entrenamiento en paralelo. Esta técnica, común en investigación científica y centros de cómputo, permitiría manejar datasets más grandes y entrenar modelos más complejos sin que los tiempos de espera se conviertan en un obstáculo. Un cluster local también ofrece la ventaja de independencia respecto a servicios

en la nube, lo cual resulta atractivo para proyectos que requieren privacidad de los datos y autonomía de infraestructura.

Integración con servicios en la nube

A pesar de las ventajas de la infraestructura local, se reconoce que la nube sigue siendo una alternativa eficiente y flexible. En particular, Google Colab Pro+ ofrece acceso a GPUs de alto nivel como NVIDIA A100 o V100, que superan ampliamente la capacidad de una GPU individual de escritorio. Con Colab se puede montar rápidamente un cuaderno Jupyter, cargar el dataset de tortugas y entrenar el modelo siamesa en cuestión de horas, en lugar de días, logrando un ahorro significativo en tiempo de cómputo. La estrategia ideal contempla un esquema híbrido: usar la GPU local para experimentos, pruebas preliminares y desarrollo continuo, mientras que los entrenamientos más pesados se ejecutan en Colab Pro+, reduciendo el tiempo total de implementación y asegurando la disponibilidad de recursos de alto rendimiento bajo demanda.

Preparación del dataset especializado

El éxito de una red siamesa depende de la calidad del dataset. Gracias al esfuerzo de documentación realizado durante más de cinco años en el hotel Moon Palace, se dispone de miles de fotografías de tortugas que han arribado de forma recurrente a las playas. Estas imágenes se encuentran organizadas por individuo, lo que facilita la generación de pares positivos y negativos para el entrenamiento. La fase 2 se concentrará en la limpieza, balance y aumento de este dataset, asegurando que cada tortuga tenga un número representativo de ejemplos para robustecer el modelo. La ventaja clave de esta base de datos es que refleja condiciones reales de campo (iluminación variable, ángulos distintos, diferencias en la arena y agua), lo cual ayudará a entrenar un modelo más realista y generalizable.

Impacto y proyección

La implementación de redes neuronales siamesas en combinación con hardware optimizado y recursos en la nube marca un paso fundamental hacia la consolidación de un sistema confiable para la identificación individual de tortugas marinas. Este método supera las prácticas tradicionales de etiquetado físico en el caparazón, que son invasivas, poco sostenibles y en muchos casos no viables en proyectos de conservación. Con el sistema propuesto, bastará con tomar una fotografía de la cabeza o las aletas de una tortuga al momento de su llegada para verificar si ya existe en la base de datos y dar seguimiento a su historial de avistamientos.

La fase 2 no solo representa un avance tecnológico, sino también una herramienta de gran valor para la conservación y la investigación científica, ya que permitirá generar registros longitudinales de individuos a lo largo de los años, facilitando el estudio de patrones migratorios, tasas de retorno y dinámicas poblacionales. En fases posteriores, este sistema podría ampliarse a una aplicación web con interfaz amigable para biólogos y voluntarios, integrando reconocimiento en tiempo real y almacenamiento en bases de datos distribuidas para trabajo colaborativo.